Servomotores Serie M17 - HOJA DE DATOS

Control de movimiento todo en uno, sencillo y asequible

De la educación a la innovación



Introducción

Los Servomotores Serie M17 son soluciones de control de movimiento todo en uno que integran un motor, un controlador de motor (driver), un controlador de movimiento y un encoder en un único paquete compacto. Estos servomotores cuentan con una interfaz de comunicación RS-485, lo que permite conectar varias unidades en cadena (daisy-chain) y controlarlas desde un único punto de conexión. Disponible en cuatro modelos - M17-60, M17-48, M17-40 y M17-34 -, la serie ofrece opciones flexibles para satisfacer requisitos específicos de par y tamaño, manteniendo características de control uniformes. Cada Servomotor Serie M17 incorpora funciones de control avanzadas, incluidos múltiples modos de operación, capacidad de autocalibración y monitoreo de estado integrado mediante indicadores LED. Los motores pueden controlarse fácilmente desde cualquier plataforma, incluidos Mac, PC, Raspberry Pi o Arduino (solo se requiere un adaptador RS485 de bajo costo), lo que los hace ideales tanto para aplicaciones educativas como industriales. La serie admite control de posición preciso con perfiles de movimiento trapezoidales, modo de control en lazo cerrado y un manejo integral de errores. Con dimensiones de montaje estandarizadas NEMA 17, un amplio rango de tensión (12-24V) y un protocolo de comunicación robusto, los Servomotores Serie M17 ofrecen una solución fiable y flexible para aplicaciones que requieren control de movimiento preciso, desde robótica y máquinas CNC hasta equipos de prueba automatizados e instrumentos científicos.

Envíenos sus comentarios

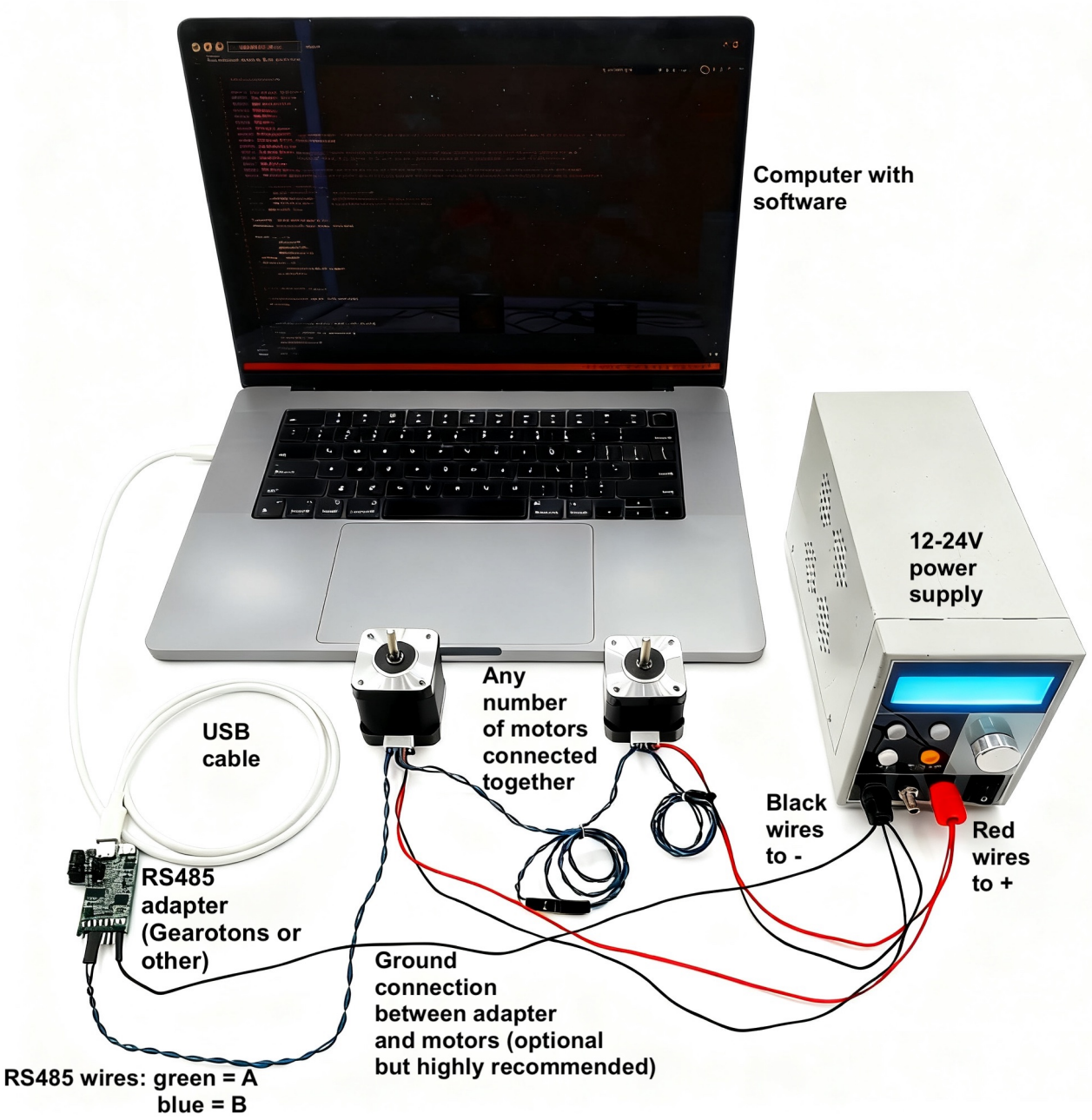
Si encuentra errores en este documento o tiene preguntas, háganoslo saber en: tutorial.gearotons.com/feedback. Agradeceremos mucho sus sugerencias sobre cómo podemos mejorar el producto o la documentación. ¡Muchas gracias!

Visite nuestra página de comentarios

Características principales

- Alto nivel de integración que combina un motor, un controlador de motor (driver), un sistema de control de movimiento y un encoder
- Controle cualquier cantidad de motores desde un único controlador sencillo a través de una interfaz de comunicación RS-485
- Control mediante comandos de alto nivel como "Enable mosfets" (habilitar MOSFETs) y "Trapezoid move" (movimiento trapezoidal), evitando el control DIR/STEP con temporización crítica
- Formato compacto de tamaño casi idéntico al de un motor paso a paso NEMA 17 con las mismas especificaciones (sin protuberancias)
- Dimensiones de montaje estandarizadas NEMA 17
- Amplio rango de tensión (12-24V) para opciones de alimentación flexibles
- Control en lazo cerrado de alta precisión con encoder integrado y lazo de control PID que se ejecuta a 32 kHz
- Mucho más eficiente energéticamente en control de lazo cerrado que un motor paso a paso convencional
- Protección integrada contra sobrecorriente, sobretensión y sobretemperatura
- La velocidad máxima puede alcanzar 560 RPM
- La relación par-peso es la misma que la de un motor paso a paso equivalente
- Compatible con una amplia variedad de interfaces y hardware, como Raspberry Pi, Arduino, ESP32, Mac y PC
- Nos esforzamos por ofrecer documentación y tutoriales excelentes para que pueda empezar a trabajar rápidamente
- Ofrecemos documentación compatible con IA por si desea que su IA favorita haga todo el trabajo por usted
- Adecuado para robótica, CNC, automatización, instrumentos científicos, útiles de prueba, impresoras 3D y mucho más
- Disponible en distintos tamaños para que pueda encontrar el par y el precio adecuados para su aplicación

Diagrama de conexión



Sistema de unidades

Los Servomotores Serie M17 utilizan ciertas unidades internas para poder realizar de forma eficiente los cálculos asociados al movimiento (mediante aritmética de enteros). Es responsabilidad del software de control admitir múltiples unidades de medida para las distintas magnitudes. Nuestras bibliotecas de Python y Arduino gestionan las conversiones de unidades automáticamente, lo que le permite trabajar con sus unidades preferidas. A continuación se indican las unidades admitidas para cada magnitud:

Magnitud	Unidades disponibles
Tiempo	seconds, milliseconds, minutes, microseconds, timesteps
Posición	shaft rotations, degrees, radians, encoder counts
Velocidad	rotations per second, rpm, degrees per second, radians per second, counts per second, counts per timestep
Aceleración	rotations per second squared, rpm per second, degrees per second squared, radians per second squared, counts per second squared, counts per timestep squared
Corriente	internal current units, milliamps, amps
Tensión	volts, millivolts
Temperatura	celsius, fahrenheit, kelvin

Guía de inicio rápido

Para ayudarle a comenzar con su Servomotor Serie M17, ofrecemos una guía en línea completa que abarca desde la configuración inicial hasta implementaciones avanzadas del protocolo. Esta guía incluye:

- Instrucciones de configuración paso a paso
- Documentación detallada del protocolo de comunicación
- Ejemplos de programación y fragmentos de código
- Descripción de los códigos de error
- Consejos de solución de problemas y buenas prácticas

Haga clic aquí para ver la guía de inicio: tutorial.gearotons.com

LEDs indicadores y botones

El servomotor tiene dos LEDs de estado (verde y rojo). El LED verde parpadea lentamente para mostrar un latido (heartbeat), parpadea rápidamente para indicar que se está ejecutando el bootloader en lugar de la aplicación, y también se enciende brevemente mientras se recibe un paquete, por lo que parpadea con la comunicación en el bus. El LED rojo indica los códigos de error fatal parpadeando un determinado número de veces.

El servomotor tiene dos botones etiquetados "Reset" (reinicio) y "Test" (prueba). El botón Reset reinicia el microcontrolador interno y todo el estado vuelve a los valores predeterminados. El botón Test hace que el motor gire. Presiónelo brevemente para que gire en un sentido, y presiónelo durante más de 0.3 segundos y suéltelo para que gire en el sentido contrario. Manténgalo presionado al menos 2 segundos y suéltelo para que el motor pase al modo de lazo cerrado. Manténgalo presionado durante más de 15 segundos y suéltelo para que el motor realice una calibración sobre sí mismo. Tenga en cuenta que el motor girará durante la calibración y debe poder girar libremente para que esta sea exitosa, por lo que debe retirar cualquier carga antes de realizar esta operación.

Descripción general del protocolo de comunicación

Los Servomotores Serie M17 utilizan RS-485 para la comunicación, lo que permite conectar varios motores en cadena (daisy-chain). Cada motor tiene una dirección única de 64 bits asignada de fábrica, de modo que puede direccionarse individualmente en un bus RS-485. A cada motor también se le puede asignar un alias único de 1 byte (0-251) para ahorrar bytes en el paquete de comunicación manteniendo un control individual similar. Existe además un alias de difusión (broadcast), que es el 255. El protocolo admite un conjunto completo de comandos para control de movimiento, configuración y monitoreo de estado. Se admite la actualización del firmware a través de la interfaz RS-485. La velocidad de comunicación está fijada en 230400 baud.

Resumen de referencia de comandos

Para consultar la fuente de referencia actualizada de todos los comandos disponibles, puede revisar este documento.

 https://github.com/tomrodinger/servomotor/blob/main/python_programs/servomotor/motor_commands.json

También puede ejecutar este comando:

```
pip3 install servomotor   # run this just once to install the library and programs
servomotor_command -c
```

Esto imprimirá la información contenida en el archivo motor_commands.json de una forma más legible y proporcionará información de uso para enviar comandos al motor desde la línea de comandos.

Control básico

Comando	Descripción
Disable MOSFETs	Deshabilita de inmediato las salidas del controlador del motor (MOSFETs); el comando se ejecuta al instante (no se encola) y es idempotente.
Enable MOSFETs	Habilita las salidas del controlador del motor (MOSFETs).
Reset time	Poner a cero el reloj absoluto de microsegundos del dispositivo; como efecto secundario importante, se vacía toda la cola de movimientos y se detiene el motor al instante.
Emergency stop	Detener el motor inmediatamente: desactivar los MOSFETs, vaciar la cola de movimientos, poner a cero la velocidad actual y ajustar la consigna interna de posición a la posición actual.
Zero position	Establece la posición actual como la posición cero (el origen).
System reset	Reinicia el dispositivo.

Control de movimiento

Comando	Descripción
Trapezoid move	Recorre el desplazamiento con signo indicado durante exactamente la duración indicada, relativo a la posición al final del movimiento previamente encolado (para un objetivo absoluto utilice 'Go to position').
Go to position	Encola un movimiento absoluto suave.
Homing	Realiza el homing del motor moviéndolo hasta que choca con un tope físico.
Go to closed loop	Entra en el modo de control de posición en lazo cerrado.
Move with acceleration	Encola un segmento de aceleración constante: la aceleración se aplica una vez por paso de tiempo durante el número de pasos de tiempo indicado.
Move with velocity	Hace girar el motor a velocidad constante durante una duración fija.
Multimove	Encola hasta 32 movimientos en un solo comando; cada movimiento es un segmento de aceleración o un cambio instantáneo de velocidad (seleccionado por movimiento mediante la máscara de bits moveTypes) ejecutado durante su número indicado de pasos de tiempo.

Configuración

Comando	Descripción
Set maximum velocity	Establece la velocidad máxima permitida.
Set maximum acceleration	Establece la aceleración máxima utilizada para planificar todos los movimientos trapezoidales encolados posteriormente ('Go to position', 'Trapezoid move', movimientos de calibración); los elementos que ya están en la cola no se ven afectados.
Start calibration	Ejecuta una calibración completa del motor.
Set maximum motor current	Establece la corriente máxima del motor y la corriente máxima de regeneración.
Set safety limits	Establece los límites de seguridad de posición.
Test mode	Establece o activa un modo de prueba.
Set PID constants	Establece las constantes P, I y D del controlador en lazo cerrado que intenta mantener la trayectoria de movimiento.
Set max allowable position deviation	Establecer la distancia máxima que puede desviarse la posición real del motor (medida por los sensores Hall) de la posición ordenada; superarla provoca el error fatal 45 (ERROR_POSITION_DEVIATION_TOO_LARGE).
CRC32 control	Habilita o deshabilita la capa CRC32 del protocolo para este dispositivo, en ambas direcciones.
Get PID constants	Leer una instantánea coherente de las constantes P, I y D sin procesar configuradas.

Estado y monitoreo

Comando	Descripción
Get current time	Devolver el tiempo absoluto del dispositivo como un número de 64 bits sin signo de microsegundos reales transcurridos desde el encendido o el último 'Reset time'; no es la hora del calendario.
Get n queued items	Devolver el número de elementos de la cola de movimientos como un solo byte entre 0 y 32 (la cola admite un máximo de 32 elementos).
Get hall sensor position	Obtiene la posición medida por los sensores Hall, que es la posición real del eje.
Get status	Obtiene los indicadores de estado del motor y el código de error fatal.
Control hall sensor statistics	Activa o desactiva la recopilación de estadísticas de los sensores Hall.
Get hall sensor statistics	Lee las estadísticas recopiladas de los tres sensores Hall analógicos.
Get position	Obtiene la posición ordenada (deseada) actual, es decir, la posición objetivo interna del perfil de movimiento que avanza en cada ciclo de control.
Get comprehensive position	Obtiene la posición ordenada, la posición de los sensores Hall (medida) y el conteo del encoder externo en una sola lectura.
Get supply voltage	Obtiene la tensión medida de la fuente de alimentación.
Get max PID error	Obtiene el error de posición mínimo y máximo observado por el controlador PID en lazo cerrado desde la última lectura, y luego restablece los acumuladores.
Get temperature	Obtiene la temperatura medida por un sensor analógico dedicado en la PCB del controlador del motor (no el chip del microcontrolador ni los devanados del motor).
Get debug values	Leer una instantánea de numerosos valores de diagnóstico internos.
Get communication statistics	Obtiene seis contadores de errores de comunicación y, opcionalmente, los restablece.

Gestión del dispositivo

Comando	Descripción
Time sync	Enviar la lectura del reloj del maestro al motor para que pueda ajustar su frecuencia de reloj.
Get product specs	Obtiene dos constantes del producto necesarias para los cálculos de movimiento.
Detect devices	Detecta todos los dispositivos conectados a la interfaz RS485.
Set device alias	Establece el alias de un byte del dispositivo, lo guarda en la flash no volátil y luego reinicia automáticamente el dispositivo.
Get product info	Obtiene la información del producto: código de producto (número de modelo), código de compatibilidad de firmware, versión de hardware, número de serie e ID único, devueltos como una carga útil de respuesta de 28 bytes.
Firmware upgrade	Escribe una página de 2048 bytes de firmware de aplicación en la flash.
Get product description	Obtiene una cadena breve de descripción del producto.
Get firmware version	Obtiene la versión del firmware, o la versión del bootloader si el dispositivo está ejecutando actualmente el bootloader.
Ping	Envía una carga útil de 10 bytes y el dispositivo responde de inmediato con los mismos 10 bytes (se ejecuta al instante, no se encola).
Vibrate	Hacer vibrar el motor (implementado solo en M1; sin efecto en M2, M17 y M23) alternando rápidamente la tensión PWM en lazo abierto entre +50 y -50 (amplitud fija codificada), o detener la vibración.
Identify	Identifica un motor haciendo parpadear rápidamente su LED verde: 30 parpadeos de unos 90 ms cada uno, unos 2.7 segundos en total, tras lo cual se reanuda el parpadeo de latido (heartbeat) normal de 1 segundo.

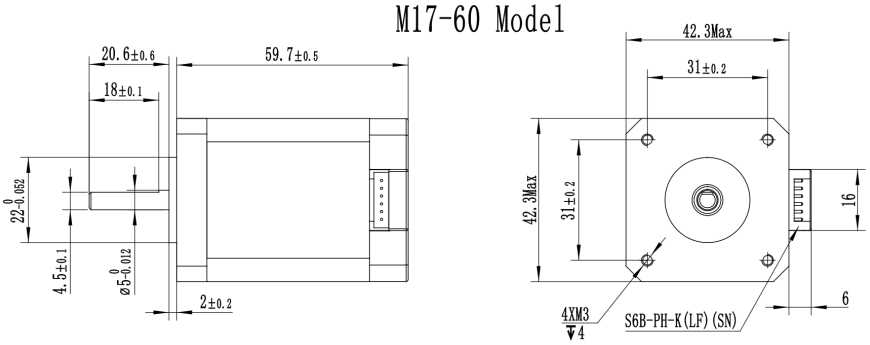
Otros

Comando	Descripción
Capture hall sensor data	Captura datos de los sensores Hall y los transmite de vuelta.
Read multipurpose buffer	Lee y borra el búfer de datos multipropósito, que contiene como máximo un conjunto de datos a la vez.

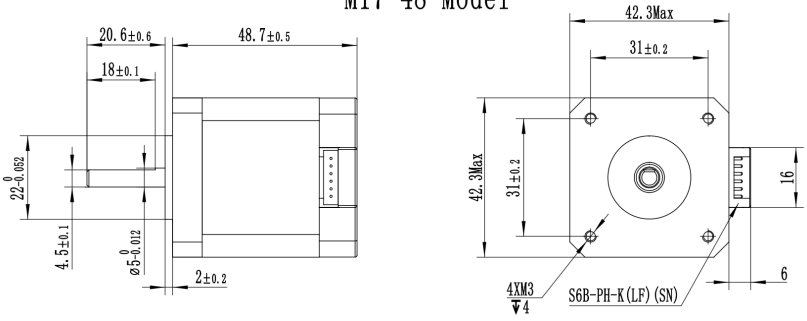
Especificaciones mecánicas

Parámetro	M17-60	M17-48	M17-40	M17-34
Dimensiones (LxA)	42.2x42.2 mm	42.2x42.2 mm	42.2x42.2 mm	42.2x42.2 mm
Altura	59.7 mm	48.7 mm	40.1 mm	33.5 mm
Longitud del eje	20.6 mm	20.6 mm	20.6 mm	20.6 mm
Peso	470g	360g	285g	210g
Clase de protección	IP20	IP20	IP20	IP20

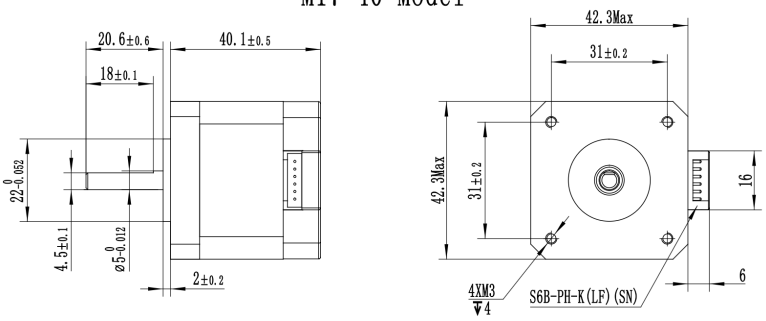
M17-60 Model



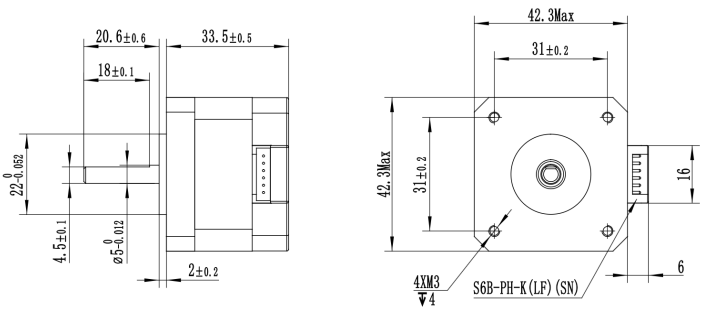
M17-48 Model



M17-40 Model



M17-34 Model



Especificaciones técnicas

Parámetro	M17-60	M17-48	M17-40	M17-34
Tensión de operación	12-24V	12-24V	12-24V	12-24V
Par nominal	0.65 N.m	0.55 N.m	0.42 N.m	0.28 N.m
Velocidad máxima	560 RPM	560 RPM	560 RPM	560 RPM
Corriente máxima	1.1A	1.1A	1.1A	1.0A
Potencia nominal	26.4W	26.4W	26.4W	24.0W

Condiciones de operación

Parámetro	Especificación
Temperatura de operación	0C a +80C
Temperatura de almacenamiento	-20C a +60C
Rango de humedad	20% a 80% RH (sin condensación)
Entorno de instalación	Solo para uso en interiores

Biblioteca de Python

Nuestra biblioteca de Python ofrece una interfaz completa para controlar los Servomotores Serie M17. La biblioteca gestiona todos los protocolos de comunicación de bajo nivel y las conversiones de unidades, lo que facilita integrar el control de servomotores en sus proyectos de Python.

Instalación

Instale la biblioteca servomotor usando pip:

```
pip3 install servomotor
```

Ejemplo: control básico del motor

El siguiente ejemplo muestra cómo conectarse a un servomotor, habilitar los MOSFETs y realizar un movimiento trapezoidal:

```
#!/usr/bin/env python3
"""
Minimal trapezoid move: rotate 1 turn in 1 second.
Edit ALIAS below if needed. Uses rotations and seconds.
"""
import time, servomotor
from servomotor import communication

# Hard-coded settings for a minimal demo
ALIAS = 'X'                               # Device alias, change if needed
SERIAL_PORT = "/dev/tty.usbserial-110"  # Serial device path; change if needed (e.g., "COM3" on
                                          #  Windows)
DISPLACEMENT_ROTATIONS = 1.0              # 1 rotation
DURATION_SECONDS = 1.0                    # 1 second
DELAY_MARGIN = 0.10                       # +10% wait margin because the motor's clock is not
                                          #  perfectly accurate

communication.serial_port = SERIAL_PORT # if you comment this out then the program
                                          #  should prompt you for the serial port or it will use
                                          #  the last used port from a file
servomotor.open_serial_port()

m = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations", verbose=0)
m.enable_mosfets()
m.trapezoid_move(DISPLACEMENT_ROTATIONS, DURATION_SECONDS)
time.sleep(DURATION_SECONDS * (1.0 + DELAY_MARGIN))
m.disable_mosfets()

servomotor.close_serial_port()
```

Características principales

- Compatibilidad con todos los comandos del motor
- Conversión automática de unidades de tiempo, posición, velocidad, aceleración, corriente, tensión y temperatura
- Manejo de errores y monitoreo de estado
- Compatibilidad con varios motores en el mismo bus
- Comience a trabajar con solo unas pocas líneas de código

Documentación

Para obtener la documentación completa de la API y más ejemplos, visite: https://github.com/tomrodinger/servomotor/tree/main/python_programs

Biblioteca de Arduino

La biblioteca de Arduino para los Servomotores Serie M17 ofrece una interfaz fácil de usar para controlar los motores desde placas Arduino. La biblioteca admite comunicación serie por hardware e incluye todas las funciones esenciales de control del motor.

Instalación

Instale la biblioteca mediante el Gestor de Bibliotecas de Arduino:

- 1. Abra el IDE de Arduino
- 2. Vaya a Programa → Incluir Librería → Administrar Bibliotecas
- 3. Busque "Servomotor"
- 4. Localice la biblioteca llamada "Servomotor by Gearotons" y haga clic en Instalar



Instalación manual

Como alternativa, si es un usuario experto, puede obtener el código de la biblioteca de Arduino aquí: https://github.com/tomrodinger/Servomotor_Arduino_Library

Ejemplo: control básico del motor

Este ejemplo muestra cómo inicializar un motor, habilitar los MOSFETs y realizar un movimiento trapezoidal:

```
// Minimal Arduino example: Trapezoid move using built-in unit conversions
// Goal: spin the motor exactly 1 rotation in 1 second, then stop.
// Sequence:
//  system reset -> enable MOSFETs -> trapezoidMove(1.0 rotations, 1.0 seconds)
//  -> wait 1.1s -> disable MOSFETs.
//
// Notes:
// - This uses the library's unit conversion (no raw counts/timesteps).
// - Configure Serial1 pins for your board (ESP32 example pins below).
// - Motor is created AFTER Serial1.begin(...) so hardware UART pins are set first.
// - Every command is checked with getError(). No command method returns a success
//   flag, and a motor that has latched a fatal error silently ignores everything
//   afterwards, so an unchecked sketch just stops moving with no symptom at all.
//   See "Checking for errors" in the API documentation.

#include <Servomotor.h>

#define ALIAS 'X'                   // Device alias
#define BAUD 230400                 // RS485 UART baud rate
#define DISPLACEMENT_ROTATIONS 1.0f // 1 rotation
#define DURATION_SECONDS 1.0f       // 1 second
#define TOLERANCE_PERCENT 10        // +10% wait margin because the motor's clock is not
                                    //  perfectly accurate
#define WAIT_MS ((unsigned long)(DURATION_SECONDS * 1000.0f * (100 + TOLERANCE_PERCENT) / 100))
#define POST_RESET_WAIT_MS 1500     // Keep the bus SILENT this long after a reset: the motor
                                    //  boots through a bootloader window, and any packet that
                                    //  arrives during it pins the motor in the bootloader.

// Example RS485 pins for ESP32 DevKit (change as needed for your board)
#if defined(ESP32)
#define RS485_TXD 4             // TX pin to RS485 transceiver
#define RS485_RXD 5             // RX pin from RS485 transceiver
#endif

// Returns true (and explains itself) if the previous command did not succeed.
// getError() is 0 on success, positive for a fatal error reported by the motor
// (look the number up in the Error Codes section of the documentation), and
// negative for a communication failure such as -1 = no reply within 1 second.
bool failed(Servomotor &motor, const char *what) {
  int e = motor.getError();
  if (e == 0) return false;
  Serial.print("[FAIL] ");
  Serial.print(what);
  Serial.print(" -> getError() = ");
  Serial.print(e);
  Serial.println(e > 0 ? "  (motor fatal error - see the Error Codes section)"
                       : "  (communication failure - check wiring, alias and power)");
  return true;
}

void setup() {
  Serial.begin(115200); // Console serial for debugging
                        // On ESP32-S3, set Tools > USB CDC On Boot > Enabled or this
                        // output never reaches the USB serial monitor.

  // Create the motor; serial port opens on first instantiation.
#if defined(ESP32)
  Servomotor motor(ALIAS, Serial1, RS485_RXD, RS485_TXD);
#else
  Servomotor motor(ALIAS, Serial1);
#endif

  // Use units: rotations for position, seconds for time.
  // These are host-side only - they send nothing to the motor.
  motor.setPositionUnit(PositionUnit::SHAFT_ROTATIONS);
  motor.setTimeUnit(TimeUnit::SECONDS);

  // Start from a known-clean state: this also clears any fatal error left over
  // from a previous run, which is the usual reason a motor "stops working".
  motor.systemReset();
  if (failed(motor, "systemReset")) return;
  delay(POST_RESET_WAIT_MS);

  motor.enableMosfets();
```

```
  if (failed(motor, "enableMosfets")) return;

  motor.trapezoidMove(DISPLACEMENT_ROTATIONS, DURATION_SECONDS);
  if (failed(motor, "trapezoidMove")) return;

  delay(WAIT_MS);

  // A successful move command only means the move was accepted and queued. Faults
  // such as a stall or a position-deviation trip happen later, while it executes,
  // so check the motor's own status once the motion should be finished.
  getStatusResponse status = motor.getStatus();
  if (failed(motor, "getStatus")) return;
  if (status.fatalErrorCode != 0) {
    Serial.print("[FAIL] motor faulted during the move, error code ");
    Serial.println(status.fatalErrorCode);
    return;
  }

  motor.disableMosfets();
  if (failed(motor, "disableMosfets")) return;

  Serial.println("Move completed successfully.");
}

void loop() {
}
```

Nuestra misión

En Gearotons creemos que el futuro pertenece a quienes comprenden la inteligencia artificial y los sistemas de automatización y pueden trabajar junto a ellos. Nuestra misión es empoderar a la próxima generación de innovadores haciendo que la tecnología avanzada de control de movimiento sea accesible, comprensible y asequible para makers, educadores e ingenieros de todo el mundo. Estamos comprometidos con derribar las barreras que tradicionalmente han separado a estudiantes y aficionados de las herramientas de automatización de nivel profesional. Al ofrecer soluciones integradas que eliminan la complejidad sin sacrificar capacidad, hacemos posibles experiencias de aprendizaje prácticas que preparan a las mentes jóvenes para un mundo donde la creatividad humana y la precisión de las máquinas trabajan en armonía. Nuestra convicción fundamental es que todo estudiante debería tener la oportunidad de experimentar con los componentes básicos de la automatización moderna, desde la robótica y los sistemas CNC hasta los instrumentos científicos. A través de nuestras alianzas educativas y nuestro enfoque de código abierto, estamos formando una generación que no solo usa la tecnología, sino que realmente la comprende y le da forma. Juntos estamos construyendo los cimientos para los innovadores que definirán un futuro asombroso.

Código abierto

Creemos en mejorar el mundo a través de la tecnología. El software, el firmware y los archivos de diseño de PCB están disponibles aquí:

https://github.com/tomrodinger/servomotor

Versión de la hoja de datos: 1.11 Fecha de publicación: Oct. 29, 2026

(c) 2026 Todas las especificaciones están sujetas a cambios sin previo aviso.